

Traitement automatique de manuscrits anciens

Computer Vision & Reconnaissance de texte manuscrit (HTR)

Master Data/IA - Module « Vision par ordinateur »

Projet réalisé en groupe de 3-4 étudiants



Projet développé par **Michel Cadennes** et **Marine Prost**

Sommaire

Sommaire	2
Contexte du projet	3
Présentation générale	3
Le pitch narratif	3
Objectifs du Volet 1	5
Périmètre fonctionnel	5
Critères d'évaluation de la qualité	5
Contraintes du projet	7
Étapes suggérées pour la résolution	9
Étape 1 - Analyse du corpus et choix des données	9
Étape 2 - Prétraitement et segmentation	9
Étape 3 - Entraînement et fine-tuning du modèle HTR	9
Étape 4 - Agrégation et évaluation finale	10
Étape 5 - Rédaction et livraison	10
Rendu final	11
i. Article scientifique	11
ii. Dépôt GitHub	11
iii. Soumission via Hetic Learn	12
Critères d'évaluation	13
Conseils et bonnes pratiques	14
Organisation de l'équipe	14
Bonnes pratiques techniques	14
Pièges à éviter	14
Ressources recommandées	14
Comment bien réussir ce projet ?	16

Contexte du projet

Présentation générale

Ce projet constitue le premier volet d'un travail en deux parties mené autour du traitement automatique de manuscrits anciens. Il mobilise l'ensemble des compétences acquises dans le module « Vision par ordinateur », depuis le prétraitement des images jusqu'à la constitution d'un corpus de transcriptions livrable au module suivant.

Le **Volet 1 - Computer Vision & HTR** porte sur la chaîne de traitement d'images et la reconnaissance de texte manuscrit (HTR, « *Handwritten Text Recognition* »). Son objectif est de produire, à partir de manuscrits numérisés, des transcriptions textuelles précises, validées par des métriques quantitatives.

Le **Volet 2 - Traitement automatique du langage** prendra en entrée les transcriptions produites par le volet 1 et les soumettra à une analyse linguistique approfondie : normalisation orthographique, annotation morpho-syntaxique, extraction d'entités nommées et modélisation thématique.

Ces deux volets sont étroitement liés : la qualité des transcriptions produites en Volet 1 conditionne directement la pertinence des analyses réalisées en Volet 2. Un pipeline mal conçu ou peu précis en amont se traduira inévitablement par des résultats NLP dégradés en aval.

Le pitch narratif

En 2026, les bibliothèques patrimoniales ont massivement investi dans la numérisation. La Bibliothèque nationale de France compte aujourd'hui environ 380 000 manuscrits dans ses collections, pour un total de collections physiques estimé à 40 millions de documents, dont un peu moins d'un quart est numérisé, soit environ 11 millions de documents accessibles en ligne via Gallica. Dans le monde, ce sont plusieurs centaines de millions de pages qui ont été converties en images haute résolution, largement diffusées grâce aux API IIF.

Mais pour un chercheur, ce travail reste inachevé : une image n'est pas un texte. Permettre la recherche plein texte, l'extraction d'entités nommées ou l'analyse statistique suppose de disposer de transcriptions. Or la transcription manuelle par des paléographes est trop lente et trop coûteuse pour passer à l'échelle. Le tarif horaire d'un-e paléographe se situe autour de 50 €/h, et le marché mondial des services de transcription dépasse 31 Md\$. La transcription manuelle d'un corpus de quelques milliers de pages reviendrait à plusieurs centaines de milliers d'euros, et à l'échelle des bibliothèques nationales, la facture serait tout simplement hors de portée. D'où le recours croissant à l'automatisation.

C'est précisément dans ce contexte qu'est née la recherche en reconnaissance automatique de l'écriture manuscrite, Handwritten Text Recognition (HTR). Porté par des consortiums de recherche en humanités numériques, ce domaine a connu des avancées spectaculaires. Citons CREMMA (Corpus pour la Recherche d'Écritures Manuscrites Médiévales en Ancien français), qui rassemble quinze manuscrits en ancien français datés du XIII^e au XV^e siècle, produits à l'aide de l'interface eScriptorium et du moteur Kraken. Plus récemment, le projet CATMuS (Consistent Approach to Transcribing Manuscript) a fédéré les équipes de CREMMA, GalliCorpora, HTRomance et DEEDS pour construire un corpus normalisé de plus de 160 000 lignes et plus de 5 millions de caractères couvrant 200 manuscrits, du VIII^e au XVII^e siècle, dans une dizaine de langues. Le catalogue HTR-United référence aujourd'hui plusieurs dizaines de jeux de données sous licence ouverte, et le projet CoMMA (début 2026) a permis de transcrire automatiquement plus de 32 000 manuscrits en quelques mois seulement à partir de modèles génériques.

Malgré ces progrès impressionnants, un verrou demeure : l'obtention de données d'entraînement de qualité. Chaque manuscrit a son écriture propre, ses abréviations, ses spécificités linguistiques. Les

bénévoles ou les projets de crowdsourcing ne peuvent fournir qu'un volume limité de vérité terrain. La BnF a exploré des dispositifs comme la plateforme CORRECT, mais la collecte reste très insuffisante face à la diversité des sources. En clair : nous ne disposons toujours pas de volumes massifs de données annotées pour entraîner des modèles robustes sur des corpus spécifiques (dialectaux, abrégés, ou à l'écriture très particulière). L'enjeu du projet est là : montrer comment, à partir d'un corpus de ground truth existant, on peut développer un pipeline complet de prétraitement, segmentation et fine-tuning pour obtenir une transcription automatique exploitable, puis discuter des limites et des besoins de données complémentaires.

Le présent projet se nourrit donc de cette réalité scientifique. Il vous met en situation réelle d'ingénierie documentaire et de computer vision : vous aurez à constituer ou sélectionner un corpus pertinent à partir des ressources libres disponibles (CREMMA Medieval, GalliCorpora, CATMuS), à implémenter un pipeline reproductible, à fine-tuner un modèle (TrOCR ou Kraken) et à livrer des transcriptions exploitables par le module NLP qui suivra. L'accent est mis sur la compréhension des biais du corpus, la gestion des incertitudes et la documentation rigoureuse, autant de compétences clés en humanités numériques et en IA de confiance.

Le programme se structure en deux phases distinctes, menées en séquence :

- **Phase 1 - HTR (Handwritten Text Recognition)** : développer une pipeline de computer vision capable de produire, à partir d'une image de page de manuscrit, une transcription textuelle fidèle.
- **Phase 2 - Analyse linguistique** : traiter les transcriptions produites en Phase 1 pour en extraire des informations structurées - entités nommées (personnes, lieux, dates), relations sémantiques, thèmes - afin d'alimenter une base de connaissances interrogeable.

Vous êtes libres du choix des architectures et des outils, dans les limites précisées dans les contraintes du projet.

Objectifs du Volet 1

Périmètre fonctionnel

À l'issue de ce premier volet, votre équipe devra avoir réalisé les éléments suivants :

1. **Constituer et préparer le corpus d'entraînement.** Sélectionner les jeux de données HTR appropriés (HTR-United, CREMMA Médiéval, CATMuS...), les nettoyer, les normaliser et les partitionner en ensembles d'entraînement, de validation et de test selon les bonnes pratiques de la section 5.1.
2. **Mettre en place un pipeline de prétraitement des images.** Implémenter la chaîne de traitement documentaire définie en cours (correction d'inclinaison, amélioration de contraste par CLAHE, binarisation adaptative) et la rendre paramétrable et reproductible.
3. **Implémenter un modèle de segmentation de structure de page (layout).** Séparer automatiquement les régions de texte, les illustrations et les marges à l'aide de SAM ou d'une architecture de segmentation dédiée (U-Net, DeepLab,...), en respectant l'ordre de lecture.
4. **Entraîner et évaluer un modèle HTR.** Fine-tuner TrOCR ou Kraken sur le corpus préparé, en suivant les protocoles d'évaluation définis en cours (CER global, amorçage pour les intervalles de confiance, test de McNemar pour la comparaison de variantes).
5. **Agréger les transcriptions et constituer le jeu de données de sortie.** Produire un jeu de données JSON validé par le schéma du « *data contract* » (section 4.4 du cours), avec scores de confiance calibrés et marquages « *needs_review* » pour les lignes incertaines.
6. **Calculer et exporter les polygones de segmentation.** Pour chaque ligne de texte détectée, votre pipeline devra produire les polygones (ou boîtes englobantes orientées) correspondant précisément au contour de la région de texte. Ces informations de structure spatiale sont essentielles pour la réutilisation par la communauté des humanités numériques (eScriptorium, Kraken, logiciels d'annotation collaborative). Le format de sortie doit être compatible avec le PAGE XML (standard pour l'encodage de la structure de page) ou, à défaut, documenté dans le data contract comme une extension normalisée.
7. **Documenter et livrer le pipeline.** Fournir un article scientifique reproductible, un dépôt GitHub documenté, et une présentation des résultats.

Critères d'évaluation de la qualité

La qualité des transcriptions sera évaluée quantitativement selon les métriques suivantes. Les seuils ci-dessous sont des objectifs progressifs : le seuil de validation est CER < 15 %, le seuil d'excellence est CER < 8 %.

Métrique	Description	Seuil de validation	Seuil d'excellence
CER global	<i>Character Error Rate</i> (distance de Levenshtein / longueur référence). Métrique principale.	< 15 %	< 8 %
WER global	Word Error Rate. Métrique complémentaire, plus sensible aux erreurs lexicales.	< 25 %	< 15 %
IC bootstrap (95 %)	Intervalle de confiance sur le CER, calculé par ré-échantillonnage bootstrap (N = 1000).	Reporté	Reporté
IAA (plafond)	CER inter-annotateurs humains sur le corpus de test, mesurant	Mesuré et commenté	CER modèle < IAA

Métrique	Description	Seuil de validation	Seuil d'excellence
	l'ambiguïté irréductible.		
Taux needs_review	Fraction des lignes transcrites marquées comme incertaines par le pipeline.	< 30 %	< 20 %
IoU segmentation	Intersection over Union entre les polygones prédits et une référence partielle (sur au moins 50 lignes fournies avec le sujet).	> 0,75	> 0,85

Toutes les évaluations sont réalisées sur le jeu de données **de test scellé**, jamais vu pendant le développement. Les équipes doivent fournir le hachage SHA-256 de leur jeu de données d'entraînement pour garantir la non-contamination. Aucun résultat obtenu en regardant le test set avant la date de rendu ne sera pris en compte.

Contraintes du projet

Les contraintes suivantes sont imposées à toutes les équipes candidates. Leur non-respect entraîne une pénalisation à l'évaluation.

1. Pipeline de bout en bout

Le projet doit couvrir l'intégralité de la chaîne de traitement, depuis la numérisation brute (images TIFF ou JPEG issues de Gallica, HTR-United, ou une autre source adaptée) jusqu'au jeu de données JSON livrable au module NLP. Un projet partiel qui couvrirait uniquement la transcription sans le prétraitement, ou uniquement la segmentation sans la transcription, ne sera pas évalué.

2. Travail en équipe

Le projet est réalisé par des groupes de **3 à 4 étudiants**. Chaque groupe désigne un responsable technique et un responsable de documentation. La répartition des tâches doit être documentée dans le README du dépôt GitHub (section « Organisation de l'équipe »). Tous les membres du groupe doivent être capables de présenter et de défendre n'importe quelle partie du projet lors de l'évaluation.

3. Reproductibilité

Les résultats doivent être entièrement reproductibles par un tiers disposant du dépôt GitHub et d'un accès à Internet. Cela implique: un seed fixé pour toutes les sources d'aléatoire, versions de toutes les dépendances spécifiées dans un fichier **pyproject.toml** ou **requirements.txt** figé, données versionnées (DVC ou hachage SHA-256 des fichiers), et instructions de reproduction dans le README (section « Reproduire les résultats »).

4. Documentation technique

Chaque fonction et classe du code doit être documentée par une **docstring** au format Google (Args, Returns, Raises, Example). Les choix d'architecture non triviaux doivent être justifiés par des commentaires dans le code. Un fichier **CONVENTIONS_TRANSCRIPTION.md** décrit les choix éditoriaux (traitement des abréviations, encodage des lacunes, conventions de casse...).

5. Tests automatisés

Le dépôt doit contenir une suite de tests **pytest** couvrant a minima : les transformations du prétraitement (formes, types, plages de valeurs), la cohérence du schéma JSON du data contract, et un test de non-régression sur le CER (le pipeline doit atteindre le seuil de validation sur un sous-ensemble de référence fourni avec le sujet).

6. Exigence de réutilisabilité des données de segmentation

Le jeu de données de sortie doit inclure, pour chaque ligne transcrite, les coordonnées spatiales (polygone ou boîte englobante orientée) permettant de localiser la ligne sur l'image source. Les formats suivants sont acceptés : PAGE XML (conforme à la dernière version du schéma), ALTO XML, ou JSON avec schéma documenté. Dans tous les cas, un fichier de métadonnées doit décrire le système de coordonnées (origine en haut à gauche, unité en pixels).

7. Licences et droits

Seuls des corpus sous licence libre (CC-BY, CC-BY-SA, domaine public) peuvent être utilisés pour l'entraînement. Les sources des images et des transcriptions doivent être citées dans le fichier **DATA_SOURCES.md** du dépôt. Les modèles pré-entraînés utilisés (SAM, TrOCR...) doivent être compatibles avec un usage de recherche non commerciale.

8. Éthique et représentativité

L'article scientifique doit comporter une section dédiée aux biais de représentation du corpus : quels siècles, régions, types de documents et profils de copistes sont surreprésentés ou sous-représentés ? Quelles

conséquences cela a-t-il sur la portée et les limitations du modèle entraîné ?

Étapes suggérées pour la résolution

Les étapes ci-dessous constituent une démarche recommandée, pas un cahier des charges rigide. Les équipes sont libres d'adapter leur approche du moment que le résultat final respecte les contraintes et atteint les objectifs de qualité.

Étape 1 - Analyse du corpus et choix des données

- **Explorer les jeux de données disponibles** : HTR-United, CREMMA Médiéval, CATMuS Medieval (HuggingFace), GalliCorpora, e-NDP. Identifier ceux qui correspondent au périmètre du projet (XIe-XVIIe siècle, vieux/moyen français).
- **Constituer le corpus de travail** : sélectionner 3 à 5 jeux de données complémentaires couvrant plusieurs siècles, régions dialectales et types de documents. Documenter les licences.
- **Analyser la distribution du corpus** : distribution par siècle, type d'écriture, longueur moyenne des lignes, taux d'abréviations. Identifier les déséquilibres potentiels.
- **Fixer les conventions éditoriales** : choisir le niveau de transcription (diplomatique, semi-diplomatique), les conventions de traitement des abréviations et des lacunes, et les documenter dès maintenant.

Étape 2 - Prétraitement et segmentation

- **Implémenter le pipeline de prétraitement** : correction d'inclinaison, CLAHE, binarisation adaptative (Sauvola), gestion des zones enluminées. Tester sur les trois types de pages (texte seul, deux colonnes, registre tabulaire).
- **Mettre en place la segmentation de structure de page** : utiliser SAM ou dhSegment pour détecter les zones texte/illustration/marge. Évaluer la qualité de l'ordre de lecture sur les pages multi-colonnes.
- **Segmenter les lignes de texte** : utiliser Kraken BLLA pour extraire les lignes de base et les polygones de chaque ligne. Vérifier la qualité de segmentation sur les trois types de document.
- **Constituer le split train/val/test** : séparer les données avant tout développement de modèle. Sceller le test set avec un hachage SHA-256.
- **Exporter les polygones dès la segmentation**. Après avoir extrait les lignes de base et les polygones via Kraken BLLA ou tout autre segmenteur, le pipeline doit immédiatement sauvegarder ces géométries dans une structure intermédiaire (par exemple un fichier .page.xml ou un dictionnaire associé à chaque ligne). Cette information sera ensuite intégrée au JSON final lors de l'agrégation. Ne pas attendre la fin du projet pour produire les polygones : leur validation précoce évite les mauvaises surprises.

Étape 3 - Entraînement et fine-tuning du modèle HTR

- **Évaluation de la ligne de base zéro fine-tuning** : mesurer le CER de TrOCR et de Kraken sans fine-tuning. Cette ligne de base est le point de référence minimal.
- **Fine-tuning TrOCR par LoRA** : fine-tuner TrOCR-base-handwritten sur le corpus préparé avec LoRA ($r = 8$ puis 16). Utiliser le jeu de données de validation pour l'arrêt prématuré de l'entraînement et la sélection du meilleur checkpoint.
- **Fine-tuning Kraken** : utiliser ketos train pour fine-tuner un modèle Kraken médiéval existant (HTR-United). Comparer les performances avec TrOCR sur le jeu de données de validation.
- **Ablation et optimisation** : tester l'impact des augmentations de données (déformation élastique, variations de contraste, bruit...). Sélectionner la meilleure configuration par ablation sur la validation.

Étape 4 - Agrégation et évaluation finale

- **Agrégation des transcriptions** : si les deux pipelines (TrOCR + Kraken) sont opérationnels, implémenter le vote pondéré par Needleman-Wunsch. Calibrer les scores de confiance.
- **Contrôle de cohérence géométrique**: Vérifiez que chaque ligne transcrite possède un polygone associé et que les polygones sont bien dans les limites de l'image. Calculez l'IoU avec le jeu de référence fourni par l'équipe pédagogique (disponible sur la plateforme du cours). Si la qualité géométrique est insuffisante, ajustez votre segmentation (post-traitement, suppression des polygones trop petits, fusion des lignes fragmentées).
- **Évaluation sur le test scellé** : calculer le CER global, le WER, les intervalles de confiance bootstrap ($N = 1000$) et le test de McNemar si deux variantes sont comparées. Cette évaluation n'est réalisée qu'une seule fois.
- **Gestion des cas limites** : identifier et flaguer les lignes très courtes, les confidences basses, les discordances inter-modèles. Calculer le taux `needs_review`.
- **Constituer le jeu de données de sortie** : produire les fichiers JSON validés par le schéma du data contract, le DatasetDict HuggingFace versionné et les fichiers de documentation.

Étape 5 - Rédaction et livraison

- **Rédiger l'article scientifique** : suivre le plan détaillé en section Rendu final. La rédaction doit être commencée dès la semaine 2, pas seulement en fin de projet.
- **Finaliser le dépôt GitHub** : s'assurer que les tests passent, que le README permet une reproduction complète, et que la model card est à jour.
- **Partager le/les modèles sur HuggingFace** avec un README, une documentation et des paramètres complétés. L'objectif étant de faire avancer le milieu de la recherche en HTR.
- **Préparer la présentation** : 8 à 10 transparents, structure recommandée en section 5.5 du cours.
- **Soumettre via Hetic Learn** : déposer avant la date limite. Tout rendu tardif entraîne une pénalisation sur la note finale.

Rendu final

Le rendu se compose de trois éléments indissociables, soumis simultanément via la plateforme Hetic Learn avant la date limite communiquée par l'équipe pédagogique. **Un rendu incomplet ou déposé hors délai entraîne une pénalisation sur la note finale.**

i. Article scientifique

L'article est rédigé selon les conventions des publications en humanités numériques et apprentissage automatique. Il doit permettre à un lecteur de **reproduire intégralement les expériences** décrites, sans contacter les auteurs. Le format de référence est celui des soumissions à ArXiv (voir arxiv.org/help/submit) : 8 à 12 pages approximativement, police 11 points, interligne simple, marges de 2 cm.

L'article doit comporter les sections suivantes :

- **Résumé (Abstract, ~200 mots) :** problème adressé, approche, résultats quantitatifs principaux (CER, WER), conclusion.
- **Introduction :** contexte de la transcription automatique de manuscrits, motivations du projet, contributions principales.
- **État de l'art :** architectures HTR (Kraken, TrOCR, DINO, SAM), jeux de données médiévaux (CREMMA, CATMuS), métriques d'évaluation (CER, WER, IAA).
- **Données :** description du corpus (sources, volumes, périodes, types d'écritures), conventions de transcription, procédure de constitution des splits.
- **Méthodes :** pipeline complet (prétraitement, segmentation, HTR, agrégation), choix d'architecture justifiés, hyperparamètres.
- **Résultats :** CER et WER sur le test scellé avec IC bootstrap, courbes d'apprentissage, ablation des composants, analyse des erreurs résiduelles.
- **Discussion :** biais de représentation du corpus, limitations du pipeline, perspectives d'amélioration.
- **Conclusion et travaux futurs :** résumé des contributions, lien avec le Volet 2 (analyse linguistique).
- **Références :** bibliographie complète au format APA ou BibTeX.
- **Annexes :** exemples de transcriptions (image + résultat), courbes de calibration des scores de confiance, description complète du data contract.

À propos de la longueur : l'important est la qualité de l'argumentation et la complétude des informations nécessaires à la reproductibilité. Un article de 7 pages très dense et bien structuré sera mieux évalué qu'un article de 12 pages bavard ou redondant. Utilisez les annexes pour les éléments volumineux (exemples, données complémentaires).

ii. Dépôt GitHub

Le code est déposé dans un dépôt GitHub public dont le nom doit être explicite et réutilisable par la communauté des humanités numériques.

La convention recommandée est : *htr-[nom_corpus_ou_manuscrit]-[année]*

Exemples : htr-cremma-medieval-2026, htr-catmus-french-2026, htr-bnf-ms-fr-12483-2026.

La structure du dépôt doit respecter les conventions documentées dans les sections 5.3 et 5.5 du cours.

Si votre groupe travaille sur plusieurs corpus, choisissez un nom générique mais précis, par exemple htr-medieval-manuscripts-XVe. Le README devra clairement indiquer l'équipe (membres, établissement) et le contexte du projet (module MD5, Vision par ordinateur).

Le dépôt doit impérativement contenir :

- README.md : guide de reproduction des résultats (installation, exécution, résultats attendus)
- pyproject.toml ou requirements.txt figé : toutes les dépendances avec leurs versions exactes
- src/ : code source documenté (docstrings, type hints)
- tests/ : suite de tests pytest opérationnelle
- experiments/journal.jsonl : journal de toutes les expériences menées
- dataset_nlp/ : jeu de données JSON de transcriptions validé par le schéma (incluant les polygones ou lien vers PAGE XML).
- MODEL_CARD.md : performances, limitations, données d'entraînement
- CONVENTIONS_TRANSCRIPTION.md : choix éditoriaux documentés
- DATA_SOURCES.md : sources, licences, attribution
- segmentations/ (ou page_xml/) : fichiers PAGE XML ou JSON de polygones.

iii. Soumission via Hetic Learn

La soumission finale est effectuée via la plateforme **Hetic Learn**.

Le nom du projet soumis doit respecter la convention : **[groupe]_htr_manuscrits_MD5.zip**. Exemple : **groupe_A_htr_manuscrits_MD5.zip**.

Critères d'évaluation

L'évaluation porte sur cinq dimensions, chacune notée sur 20. La note finale est la moyenne pondérée.

Dimension	Ce qui est évalué	Poids
Performance HTR	CER et WER sur le test scellé, intervalles de confiance, comparaison avec la baseline, analyse des erreurs.	30 %
Qualité du pipeline	Complétude du pipeline (du scan brut au JSON livrable), reproductibilité, tests passants, gestion des cas limites.	25 %
Article scientifique	Clarté, rigueur, exhaustivité, qualité de la bibliographie, section de reproduction de l'expérience.	20 %
Documentation & code	Qualité des docstrings, README, model card, conventions éditoriales, organisation du dépôt GitHub.	15 %
Présentation orale	Clarté des slides, justification des choix, honnêteté sur les limitations, maîtrise des réponses aux questions.	10 %

Points de bonus possibles (jusqu'à +3 points sur la note finale) :

- **Comparaison de deux architectures HTR distinctes** (ex. TrOCR vs Kraken) avec test de McNemar et analyse qualitative des erreurs différentielles. (+1 point)
- **Section éthique approfondie** comprenant une mesure de l'IAA sur le corpus, une analyse des biais de représentation dialectale et sociale, et des recommandations concrètes. (+1 point)
- **Intégration eScriptorium** : import du PAGE XML Kraken dans une instance eScriptorium et démonstration d'un flux de correction humaine. (+1 point)

Conseils et bonnes pratiques

Organisation de l'équipe

- **Désigner des rôles dès le premier jour.** Un responsable technique (merge requests, revue de code), un responsable de documentation (article, README), un responsable de l'expérimentation (journal des essais, métriques) et un responsable de la gestion des données (jeux de données, licences, data contract).
- **Tenir des réunions courtes et régulières.** Un stand-up de 15 minutes par jour de travail, et une rétrospective hebdomadaire de 30 minutes où chacun présente ce qu'il a fait, les blocages rencontrés et les prochaines étapes.
- **Utiliser une branche par fonctionnalité.** Ne jamais travailler directement sur main. Chaque merge request doit être relue par au moins un autre membre du groupe.
- **Ne pas attendre la fin pour rédiger.** L'article doit être commencé en semaine 1 (introduction, état de l'art, données). Les résultats s'ajoutent au fil des expériences.

Bonnes pratiques techniques

- **Fixer les seeds dès le départ.** Une seule fonction `fixer_seeds(42)` appelée en début de chaque script. Les résultats non reproductibles ne peuvent pas être défendus.
- **Séparer le test set avant de commencer.** Créer le split train/val/test le premier jour et ne jamais regarder le test set pendant le développement. Calculer et enregistrer son hash SHA-256.
- **Évaluer souvent sur la validation.** Ne pas attendre la fin pour savoir si le modèle converge. Des courbes d'apprentissage (CER val en fonction des epochs) doivent être produites à chaque entraînement.
- **Documenter les expériences en temps réel.** Chaque run d'entraînement doit être enregistré dans le journal d'expériences (hyperparamètres, CER val, durée, checkpoints). Un résultat non tracé est un résultat perdu.
- **Commencer simple, complexifier ensuite.** Obtenir une baseline fonctionnelle end-to-end en semaine 1 (même avec un CER de 30 %). Il vaut mieux un pipeline entier médiocre qu'un composant parfait et un pipeline incomplet.

Pièges à éviter

- **Ne pas optimiser sur le test set.** Évaluer sur le test scellé une seule fois, pour le rendu final. Choisir les hyperparamètres uniquement sur la validation.
- **Ne pas négliger le prétraitement.** La qualité des images en entrée conditionne directement le CER. Une mauvaise binarisation peut dégrader le CER de 5 à 10 points.
- **Ne pas ignorer les cas limites.** Les lignes courtes, les zones dégradées et les mélange de langues représentent une fraction significative du corpus. Les identifier et les traiter explicitement (flags `needs_review`) est attendu.
- **Ne pas sous-estimer la rédaction.** Un article de 10 pages rigoureux demande autant de travail que le code. Commencer tôt, et faire relire par des membres du groupe qui n'ont pas écrit la section.
- **Ne pas oublier les licences.** Un corpus utilisé sans vérification de licence peut invalider la publication ou le dépôt GitHub. Vérifier chaque source dès le début.

Ressources recommandées

Datasets : HTR-United (github.com/HTR-United), CATMuS, Medieval

(huggingface.co/datasets/CATMuS/medieval), CREMMA Médiéval, e-NDP (Zenodo), HIMANIS-Guérin (Zenodo).

Modèles : microsoft/trocr-base-handwritten (HuggingFace), facebook/dinov2-base, Kraken BLLA + modèles HTR-United, segment-anything (Meta).

Bibliothèques : transformers, peft, jeux de données (HuggingFace), kraken, opencv-python, scikit-image, editdistance, jsonschema, segmentation-models-pytorch.

Documentation eScriptorium : escriptorium.fr - **Documentation Kraken** : kraken.re - **HTR-United** : htr-united.github.io

Comment bien réussir ce projet ?

Ce projet est ambitieux. Il demande de combiner des compétences en traitement d'images, en apprentissage profond, en ingénierie logicielle, en documentation scientifique et en travail d'équipe. Voici quelques principes qui distinguent les groupes qui réussissent de ceux qui peinent.

Le pipeline avant la perfection

La tentation est grande de passer trois semaines à fine-tuner le meilleur modèle possible avant de s'occuper de la segmentation ou du data contract. Résistez à cette tentation. **Un pipeline end-to-end avec un CER de 20 % vaut plus qu'un modèle parfait sans pipeline.** Le jury évalue l'ensemble du pipeline, pas un seul composant.

La documentation est du code

Une fonction sans docstring est une fonction à moitié écrite. Un README qui ne permet pas de reproduire les résultats est un README inutile. **Documentez au fur et à mesure, pas en fin de projet.** Imaginez qu'un membre du groupe tombe malade la veille du rendu - le code doit être compréhensible par quelqu'un qui ne l'a pas écrit.

Cherchez, questionnez, itérez

Le champ de la reconnaissance de texte manuscrit est actif et bien documenté. Les articles des challenges ICDAR, les discussions sur le forum HTR-United, et la documentation de Kraken et eScriptorium contiennent des réponses à la plupart des problèmes que vous rencontrerez. **Posez des questions, consultez la littérature, testez des hypothèses.** C'est ainsi qu'on apprend ce métier.

Soyez honnêtes sur les limitations

Un article scientifique qui décrit uniquement les succès et passe sous silence les échecs est un mauvais article. **Discuter franchement des limitations** - corpus non représentatif, sous-performance sur certains types d'écriture, biais potentiels - est une marque de maturité scientifique appréciée par les évaluateurs.

Bonne chance à toutes et tous.